Project Folder Creator User Guide

0.4.1 build 20 · 3 September 2026

Project Folder Creator creates one numbered project from a reusable main template, with an independent folder selection for each Edit, Archive, Cloud, or Custom storage destination.

This is the canonical operator guide for Project Folder Creator 0.4.1 build 20.

> **Development status:** planning, destination-specific copy rules, nested renaming, staged creation, verification, partial-failure reporting, retry, JSON/text receipts, saved profiles and reading destinations from a CopyTrust preset are implemented. CopyTrust *handoff*, per-destination live progress, cancellation during an individual large copy, and acceptance on real facility storage are not complete. Do not treat this build as a production release.

> **New since 0.3.1 build 12:** a receipt is written to this Mac on every run and a chosen receipts folder is no longer required (0.4 build 19, and 0.4.1 build 20 where the Create button was still asking for one); a note can optionally be left in each created project folder; a profile can carry its own receipts folder; and creation is refused on storage whose volume is not really mounted, at any depth below `/Volumes` (0.3.2–0.3.7).

What the app creates

One main template can produce a different projection on each destination. For example:

- Edit storage can receive the complete project template;
- Archive storage can receive only `ln {project}` and its descendants;
- Cloud storage can receive selected collaboration and delivery folders;
- Custom storage can use its own selection and folder arrangement.

The app renders project tokens in the project folder and in selected folder or package names. It stages and verifies each destination independently, commits a successful destination with a same-volume rename, and writes a receipt describing the complete result.

Before you begin

Prepare:

1. A readable main template folder.
2. A project number in `YYYY-NNN` form, such as `2026-123`.
3. A project name, such as `Example Documentary`.
4. Every storage folder the project should be created on.
5. The existing project directory inside each storage, if projects are not kept at its root.
6. Optionally, a writable receipts folder outside the template and created projects, if the receipt kept on this Mac should also be copied somewhere the facility can see it.

Use fictional or facility-approved names while testing. Examples in this guide use paths such as `/Volumes/Example_Edit/Projects` and do not describe a real facility.

Quick start

1. Drop the main template onto **Main template**, or click the drop area and choose it.
2. Enter the project number and project name.
3. Choose **Add Edit, Archive, Cloud or Custom Storage…** for each destination.
4. Set where each storage keeps projects and whether projects are arranged directly or by year.
5. Choose **Edit Copy Rules** on each destination and select the required template branches.
6. Use the pencil beside any template name that needs a token or another created name.
7. Optionally choose a receipts folder for an extra copy of the receipt.
8. Review the read-only creation plan, including every final path and warning.
9. Choose **Create Project**, confirm, and review each destination result.
10. Reveal the receipt. If a destination failed, correct the cause and choose **Retry Incomplete**.

The Create button is disabled until the plan is valid. Step 7 is genuinely optional: a receipt is written to this Mac whether or not a folder is chosen.

If somebody has handed you a profile, steps 1, 3, 4 and 5 are already answered: choose **Profile → Import Profile…** once, and from then on the job is a project number and a project name. See *Profiles*, below.

1. Choose and inspect the main template

The template is read without modification unless you explicitly choose **Save Names With This Template**.

The inventory distinguishes:

- ordinary folders;
- ordinary files;
- symbolic links, including broken links;
- macOS packages such as `.fcpbundle` and `.workflow`.

A package is one indivisible leaf in the selection tree. It can be selected and renamed as a whole object, but its internal contents are not exposed as separate copy choices.

The app shows the template item count and fingerprint. A restored workspace whose template fingerprint changed displays **The template has changed since these copy rules were saved**. Review every destination selection. A previously selected branch that no longer exists is a planning refusal, not a silent omission.

Template exclusions

Open **Settings** → **Template** to manage exclusion patterns. `.DS_Store` is excluded by default. Patterns are tested against both the item's name and its path inside the template:

Pattern	Example result
`.DS_Store`	Excludes that name anywhere in the template
`*.tmp`	Excludes temporary files by name
`Cache/*`	Excludes matching paths inside `Cache`

Changing exclusions re-reads the template. Existing selections remain associated with their literal template paths; if an excluded branch was selected, the plan identifies the now-missing selection before creation.

`.projectfoldercreator.json` is always excluded. It is the template's optional naming sidecar, not project content.

2. Enter the project identity

The project number must exactly match `YYYY-NNN`. The project name cannot be empty. Neither value may contain `{`, `}`, `[`, or `]`, because project identity values are substituted into template names.

Supported project tokens are:

Token	Example rendering
`{project}`	`2026-123`
`{project_name}`	`Example Documentary`
`{project_full}`	`2026-123 Example Documentary`

CopyTrust ingest-only tokens are not accepted in project templates.

Existing project list

After storage is added, the app scans each configured project directory. One project number found on several destinations appears once with all locations. Different folder names for the same number are shown rather than hidden.

- Choose a project in the menu to reveal it in the Finder.
- Choose **Use YYYY-NNN** to take the next unused number in that year — the year of the number already typed, or the current year when the field is empty. It is offered only after storage has been read.
- Use the refresh button to read the storage again.
- Template-looking folders are listed separately as skipped templates.
- If a destination cannot be read, the visible project list is marked incomplete.

This list is a convenience and may become stale. The planner performs its own current collision scan before any creation and is the authority that blocks an existing project number.

3. Configure storage destinations

Adding a destination selects a volume or folder. Each destination card has four independent settings:

1. **Role** — Edit, Archive, Cloud, or Custom.
2. **Label** — the human-readable destination name recorded in the plan and receipt.
3. **Projects in** — a relative project directory such as `Projects` or `Jobs/Active`; leave blank when the chosen folder itself holds projects.
4. **Arrangement** — projects are directly in the project directory or inside a `YYYY` folder.

Examples:

```
/Volumes/Example_Edit/Projects/2026/2026-123 Example
Documentary
                                ^ year arrangement

/Volumes/Example_Cloud/Projects/2026-123 Example Documentary
                                 ^ direct arrangement
```

The path beneath each destination card is the actual project location the app will use. Orange **not there yet** text means the configured project directory does not exist.

The chosen storage folder and configured project directory must already exist and be writable. The year folder beneath the project directory may be created when needed. A missing project directory is refused so a mistyped relative path cannot create a second project tree.

The storage must also be genuinely mounted. macOS leaves the mount-point directory behind after an unclean unmount, so a folder under `/Volumes` can look completely normal in the Finder while nothing is mounted on it; a project created there is written to this Mac's own disk and disappears the moment the real volume mounts over it. The app settles this from the kernel's mount table rather than from the folder existing, and it does so at any depth — a cloud filespace that mounts two levels below `/Volumes`, with an ordinary folder as its container, is recognised correctly and the container is not. Creation on an unmounted destination is refused by name.

Two destinations may share a volume when their project directories are different. Adding the same project directory twice is refused.

Storage defaults and CopyTrust layouts

Open **Settings** → **Storage** to set the project directory and arrangement used by newly added destinations.

When CopyTrust has enforced project layouts on the Mac, this tab can apply a layout to the app's destinations. It applies only the relative directory and arrangement. The volumes remain the storage folders selected in Project Folder Creator.

To take the **volumes as well**, use **Profile** → **Take from the Preset on This Mac** or **Take from a Preset File…** instead. A preset carries its destinations portably, so that import fills in the storage, the project directory and the arrangement together. See *Destinations from a CopyTrust preset*, below.

4. Set copy rules for every destination

Choose **Edit Copy Rules** on a destination to make it active in the template tree.

- Selecting a folder selects its current descendants.
- Deselecting a folder deselects its descendants.
- Selecting only a deep file or folder creates the required parent folders without adding unselected siblings.
- A mixed checkbox means only part of that branch is selected.
- Closing a branch does not change its selection.
- **All** and **None** affect only the active destination.
- **Expand** and **Collapse** change the view, not the copy rules.

Every destination must select at least one template item.

5. Customize created names

The green preview beside a template item shows what its name becomes. Use the pencil to add or replace the exact rename rule for that template name.

For example:

```
1n 2026-xxx                          → 1n {project}
2026-xxx TEMPLATE.fcpbundle          →
{project_full}.fcpbundle
```

The default global rename rule is:

```
year-xxx → {project}
```

Rename rules run before project tokens are rendered and apply to folder, file, and package names. Open **Settings** → **Naming** to manage the complete rule list and the project-folder template. The project-folder template must contain `{project}` or `{project_full}` so project scans can find the created folder by number.

An orange **not being renamed** warning means the item still looks like a template placeholder and would be copied literally. Correct it before creating.

Save names with the template

Choose **Save Names With This Template** to write the relevant rename rules to `.projectfoldercreator.json` inside the template. A Mac that loads the template adopts those rules and reports that the names came from the template.

This is the only normal action that writes into the template. The sidecar is never displayed as a selectable item and is never copied into a project.

6. Receipts

A receipt is always kept. Every run writes matching JSON and text receipts to:

```
~/Library/Application Support/Project Folder Creator/Receipts
```

There is no setting for this and nothing to choose. Evidence of what was created should not depend on somebody having picked a folder for it, and a facility's chosen folder tends to live on storage that gets unmounted — which is exactly when a record of the run is worth most.

An extra copy, if you want one

Choose a writable folder outside the template and outside every created project, and the same two receipts are written there as well. **Clear** stops it. A copy that fails to land in the chosen folder is reported and does not fail the run: the receipt on this Mac still exists, and the app says where the receipt did land.

A note in each project folder

Off by default. Turn on *Leave a note in each project folder* to write `_ProjectFolderCreated.txt` at the top of every project created, naming the project, the template and its fingerprint, the app and version, the Mac, the operator, the moment, the profile, and the session id.

It is deliberately a pointer rather than a second receipt — the session id ties it to the full record, so the two cannot drift — and the file says so in as many words.

Leave it off unless a facility has asked for it. This app's contract is that what it creates matches the template, and an unrequested file breaks a facility's own validation and shows up in any folder comparison, Folder Copy Compare included. The note is written after verification and never fails a run: a project without its note is still a project.

7. Review the read-only plan

The lower-right panel is the authoritative preview. For each destination, review:

- the final project path;
- warnings;
- selected-item and planned-path counts;
- every rendered path under **Show the folders this creates**.

Required ancestors are included in planned paths even when only a deeper item was selected.

Warnings

Low or unknown free space is shown as an orange warning and recorded in the receipt. It does not invalidate the plan. Free space is assessed per volume because several destinations on one volume compete for the same capacity.

Planning refusals

The app refuses creation before writing when it finds:

- an invalid or empty project identity;
- project identity values containing token brackets;
- no destination or an empty destination selection;
- a duplicate, missing, or unwritable destination;
- a destination whose volume is not mounted, however ordinary its folder looks;
- a missing configured project directory;
- a selected template branch that is missing or unreadable;
- an unknown, unsupported, unresolved, or empty-rendering token;
- a project-folder template without the project number;
- two template items that render to the same destination path;
- an existing project number or final project path.

A name such as `[Archive]` may genuinely be a literal folder rather than a token. Use the offered **is a real folder name** control only after checking the template. The confirmation lasts for the current session and is recorded in the receipt.

8. Create and verify the project

After you choose **Create Project** and confirm, the executor handles each destination independently:

1. Creates a hidden staging folder beside the final project path.
2. Copies the planned entries in parent-before-child order.
3. Verifies every planned folder, file, package, or symbolic link.
4. Rejects content in the staged tree that the plan did not request.
5. Commits the staged tree to the final path with a same-volume rename.

The app never overwrites or merges an existing path. Several storage systems cannot form one atomic filesystem operation: one destination may be committed even when another later fails.

The UI reports, per destination:

- **verified** — created and checked;
- **created but unchecked** — written before a later failure stopped verification;
- **failed** — the attempted path or verification that failed;
- **not reached** — planned paths skipped after an earlier failure.

9. Read the receipts

Use **Reveal Receipt** after a run. It opens the copy in this Mac's own receipts store, which is the one that is always written. Receipt schema 4 records:

- schema, session, completion time, app version, and build;
- project number and name;
- template path and fingerprint;
- exclusions, rename rules, and optional copy-rule profile identity;
- warnings shown before confirmation;
- literal bracketed names confirmed by the operator;
- every destination's role, label, roots, rendered project folder, and selected template paths;
- every planned path, kind, selection reason, result, and error;
- retained staging locations;
- the stored all-destinations-verification verdict.

The JSON file is the machine-readable record. The text file is the matching operator summary. Both are written to every configured location — this Mac's store, and the chosen folder when there is one.

10. Recover from a partial run

A failed destination may retain its hidden staging folder. The app lists discovered staging folders in the plan area and provides a Finder reveal action.

Do not treat retained staging as a completed project. Inspect it before deciding whether to remove it. Project Folder Creator never deletes staging automatically because it may be the only record of what the failed copy achieved.

After correcting the storage or source problem, choose **Retry Incomplete**. Retry:

- replans only destinations that failed;
- does not recopy destinations that already committed;
- reverifies committed destinations at their final paths;
- writes a new receipt that still answers for the complete requested destination set.

Work legitimately added to a committed project is accepted during retry; unplanned-content checking applies to a staged tree written by this tool, not to a project people may already be using.

After a complete run, choose **New Project**. It clears the project number and name while keeping the template, destinations, copy rules, and receipts folder.

Profiles

A profile is the whole set-up under a name: the template, the naming settings, and which branches of the template each storage receives. One person answers every question in this app once, checks the answers against real storage, and hands the result to somebody who is then asked for a project number and a project name and nothing else.

The **Profile** control in the window header names the profile in force, or reads *No Profile*.

What a profile carries

- the template itself — a copy of it, inside the profile;
- exclusions, rename rules and the project folder name;
- every destination, as a volume name and a path beneath it;
- each destination's project directory and arrangement;
- which template branches each destination receives;
- optionally, a receipts folder, when the one in force sits on a volume.

It deliberately does **not** carry a project number or project name. Those belong to one job, and a profile loaded on Tuesday supplying Monday's number is how the wrong project gets created.

Why a profile is a folder rather than a file

An exported profile is a `.pfcprofile` folder holding `profile.json` and a copy of the template. The app copies files, packages and symbolic links out of a real template folder — only plain directories are created outright — so a profile that carried a description of the template rather than the template itself would reproduce an all-folders template correctly and would produce an empty `.fcpbundle` on the receiving Mac.

The consequence is the useful one: **a Mac that opens a profile needs nothing else.** No template volume, no shared folder, no CopyTrust.

Saving one

Profile → Save as Profile…, give it a name and an optional note. The template is copied into the profile as it is saved, and the app then works from that copy — so editing the master template this afternoon does not change what the profile creates.

A destination that is not on a mounted volume, such as one inside a home folder, has no portable form. Saving refuses and names it, rather than writing a path that would resolve to nothing on somebody else's Mac. Point it at the storage itself, under `/Volumes`, and save again.

Saving while a profile is loaded updates that profile rather than making a near-duplicate.

Handing one over

Profile → Export Profile… writes the `.pfcprofile` bundle. Send it however you send anything.

On the receiving Mac, **Profile → Import Profile…** copies it into that Mac's own Application Support and loads it. It is copied rather than read where it sits, because where it sits is a Downloads folder that will be tidied up. Importing the same profile again replaces it rather than adding a second copy, so a corrected profile can be redeployed to a client at any time.

What a loaded profile tells you

- **Destinations that are not mounted** are listed and nothing is blocked. The archive drive gets plugged in at ten.
- **Template drift** — if the profile's own template has been changed since the copy rules were chosen, the app says what changed. A template that only gained folders is normal and is reported quietly. A branch that a destination was told to take and which no longer exists puts a warning icon on the Profile control and names the branch.

A profile that names its own receipts folder overrides this Mac's remembered one, so a deployment can say where the extra copy goes without an operator reselecting it on every machine. A profile naming none leaves this Mac's own choice alone. As with every location in a profile, one on somebody's home folder has no portable form and is not carried: it would resolve to nothing on the next Mac, and looking configured is worse than being absent.

Work Outside This Profile changes nothing on the bench. It says only that further changes are this Mac's own rather than edits to a deployed profile.

The profile's name is written into every receipt, so a project created months ago can be read back against the set-up that produced it.

Destinations from a CopyTrust preset

A facility running CopyTrust has already described its storage once. A preset carries the volumes and the paths beneath them, and the naming convention inside it carries each storage's project directory and how far below it a project folder sits. Together that is a complete destination here, so it can be imported rather than typed a second time.

- **Profile → Take from the Preset on This Mac** — the preset currently applied on this Mac.
- **Profile → Take from a Preset File…** — a preset `.json`, which is the normal choice when setting a profile up for somebody else's facility.

Only the storage is read. Nothing about card copying is: not verification, not proxies, not P5, not the convention for delivered card folders.

What the preset cannot say is which branches of the template belong on which storage. That is this app's question. Imported destinations therefore take the whole template until you narrow them, which is the step that turns an import into a profile worth handing over.

Two things the import reports rather than leaving you to notice:

- **Proxies-only destinations are left out**, because they receive dailies rather than a project folder.
- **Where the preset has several project roots** and a destination's label matches none of them, the first root is used and that destination is named. Both labels are free text somebody typed, so "Archive RAID" is matched to the shape called "Archive"; where nothing matches, the fallback is said out loud because a wrong project directory would otherwise create projects at the top of a drive.

Building a profile from a client's preset, start to finish

This is the whole sequence for handing a client a set-up built from the CopyTrust preset they already use.

1. **Get the client's CopyTrust preset** `.json` — the same file their operators load in CopyTrust. Nothing needs to be installed to read it.
2. **Open the client's folder template** in Project Folder Creator — drop it on **Main template**. This is the folder whose contents form a project.
3. **Profile → Take from a Preset File…** and choose their preset. The destinations appear with their volumes, project directories and arrangements filled in. **Read the notice line** under the header: it says how many destinations were taken, which were left out, and any whose project directory it had to guess.
4. **Check each destination.** The role and the label are guesses from the preset's own free text; the project directory and arrangement are not. Fix the labels and roles, and correct any project directory the notice flagged.
5. **Set the copy rules.** Choose **Edit Copy Rules** on each destination and select the branches that storage should receive — the whole template for Edit, perhaps only `ln {project}` for Archive. This is the part no preset can supply, and the reason the profile is worth making.
6. **Check the naming.** Settings → the project folder name, the rename rules, and the exclusions. Use the pencil beside any template name that still reads as a placeholder.
7. **Test it before you hand it over.** Enter a real project number and name, review the plan, and create it on storage you can throw away. Read the receipt.
8. **Profile → Save as Profile…** and name it for the client.
9. **Profile → Export Profile…** to write the `.pfcprofile` bundle, and send that one folder.

On the client's Mac: **Profile → Import Profile…**, choose the bundle, and that is the whole set-up. Their operators enter a project number and a project name and choose **Create Project**. If a destination volume is not mounted, the app says so and stops nothing else.

If you correct something later, load the profile, change it, **Save Changes to Profile…**, export it again and send it. They import it and it replaces the one they had.

What the app remembers

The app restores:

- the main template path and fingerprint;
- storage destinations, labels, roles, project directories, and arrangements;
- each destination's template selection;
- the chosen receipts folder, if there is one;
- exclusions, rename rules, project-folder naming, and new-destination defaults.

It deliberately does not restore the previous project number or project name.

Where a profile is loaded, the profile is restored instead of the remembered bench: a profile is what somebody decided this Mac should be doing, and restoring yesterday's ad-hoc destinations over it would quietly undo that. The chosen receipts folder is the exception, restored on both paths, because where receipts go is a fact about this Mac rather than about the bench.

Literal bracket-name confirmations last only for the current session.

Troubleshooting

Create Project is disabled

Read the plan panel and correct the refusal it names. Every destination needs at least one selected template item. A receipts folder is not required.

The project directory says “not there yet”

Check the relative path on the destination card. Create the intended project directory outside the app if it does not exist. Only the year folder is created automatically.

The project list is incomplete

One or more destinations could not be read. Check the storage connection and permissions, then use the refresh control. Do not assume a missing row means a project is absent from unreadable storage.

A template name is orange

The name resembles an unresolved facility placeholder. Use the pencil to give it a tokenized created name, then confirm the green preview.

The app reports an unknown token

Correct mismatched or unsupported brackets. If the complete bracketed text is intentionally a literal folder name, verify it and use the offered confirmation control.

A storage is reported as not mounted

The folder exists and looks right, and nothing is mounted on it. Mount the volume and try again. The refusal prints the whole mount table beneath it — every mount point in full, the volume name each reports, and its filesystem — because a volume's name and the folder it mounts at are allowed to differ, and comparing the two is usually the answer. This is not a reason to edit the destination's path.

Receipt writing failed

The extra copy could not be written; the receipt on this Mac is unaffected and the run keeps its real verdict. Choose a writable folder outside the template and projects, or **Clear** it if the extra copy is not wanted.

A hidden staged copy is listed

Reveal and inspect it in the Finder. It is not committed project storage and is not removed by the app. Use the receipt and path outcomes to understand where the run stopped.

Current capability boundary

The following are not complete in version 0.4.1 build 20:

- CopyTrust *handoff* — a refreshable project list, or **Create missing project…** opening this workflow from inside CopyTrust. Reading a preset's destinations is implemented; handing results back is not;
- per-destination live byte/file progress;
- cancellation during an individual large copy operation;
- acceptance testing on real Edit, Archive, and Cloud facility storage;
- shared multi-operator project-request or registry coordination.

Source and verification map

This guide is backed by the current implementation and focused tests. Paths are given as text rather than links: a relative link is resolved to an absolute `file://` URI when this guide is rendered to PDF, which would write the building machine's home directory into a document that ships inside the app.

- `ProjectFolderCreatorView.swift` — app workflow and controls
- `ProjectFolderCreatorModel.swift` — application state, persistence, planning, and retry
- `ProjectFolderCreatorSettingsView.swift` — settings and CopyTrust shape import
- `ProjectFolderCreatorModel+Profiles.swift` — saving, loading, exporting and importing profiles
- `ProjectFolderCreatorProfileStore.swift` — where profiles live on this Mac
- `ProjectFolderCreatorProfileMenu.swift` — the Profile control and its notice line
- `../CopyCore/Sources/CopyCore/ProjectCreatorProfile.swift` — the profile, portable locations, and the bundle
- `../CopyCore/Sources/CopyCore/CopyTrustPresetImport.swift` — destinations read from a CopyTrust preset
- `../CopyCore/Tests/CopyCoreTests/ProjectCreatorProfileTests.swift` — profile, bundle and preset-import tests
- `../CopyCore/Sources/CopyCore/ProjectTemplate.swift` — template inventory, exclusions, packages, and naming sidecar
- `../CopyCore/Sources/CopyCore/ProjectCreation.swift` — planning, execution, verification, and receipts
- `../CopyCore/Sources/CopyCore/MountedVolumes.swift` — whether a path is really on a mounted volume
- `../CopyCore/Tests/CopyCoreTests/ProjectFolderReceiptTests.swift` — the always-written receipt and the project-folder note
- `../CopyCore/Sources/CopyCore/ProjectDirectory.swift` — existing project discovery and grouping
- `../CopyCore/Tests/CopyCoreTests/ProjectCreationTests.swift` — focused creation tests
- `../docs/PROJECT_FOLDER_CREATOR_PLAN.md` — implementation and acceptance plan

The in-app Help window is a concise companion to this guide. Open it from the **Help** button in the main window or **Help → Project Folder Creator Help**.